SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Document Information

Project Name: AI-Powered Integrated Bid Compliance Verification Platform for GeM Procurement
SIH Problem Statement ID: 26100
Document Type: Software Requirements Specification
Version: 1.0

---

2. Introduction

2.1 Purpose

This system is an AI-powered platform designed to automate and assist the verification of bidder compliance against procurement/tender requirements.

The system will analyze tender documents, extract structured requirements, process bidder-submitted documents, extract relevant information using OCR and AI, compare evidence against requirements, perform available verification checks, identify inconsistencies, and generate an explainable compliance report.

The platform must combine:

AI + deterministic rules + verification + human review + auditability

AI must assist the decision-making process but should not blindly make critical procurement decisions without supporting evidence or appropriate human review.

---

3. Scope

The system will provide the following capabilities:

1. User authentication and authorization
2. Role-based access control
3. Tender management
4. Tender document upload
5. AI-based requirement extraction
6. Structured requirement management
7. Bidder registration/submission
8. Secure document upload
9. OCR and document processing
10. Information/entity extraction
11. Requirement-document matching
12. Rule-based compliance checking

13. AI-assisted compliance analysis
14. Cross-document consistency checking
15. Certificate/date validity checking
16. External/mock verification framework
17. Risk and confidence assessment
18. Human review workflow
19. Compliance dashboard
20. Audit logging
21. Report generation
22. Notifications/status tracking

---

4. Stakeholders

4.1 Bidder

Uploads bid documents and monitors the status of submitted bids.

4.2 Procurement Officer

Creates tenders, reviews bidder submissions, examines compliance results and manages procurement workflows.

4.3 Reviewer

Reviews cases flagged by the AI/rule engine and provides final human decisions.

4.4 Auditor

Views authorized audit and verification information.

4.5 Administrator

Manages users, roles, permissions and system configuration.

---

5. User Roles and Permissions

5.1 Admin

Permissions:

- Create users
- Disable users
- Assign roles
- Modify system configuration

- View audit logs
- Manage verification providers
- Manage system settings

5.2 Procurement Officer

Permissions:

- Create tender
- Upload tender
- Edit tender
- Publish tender
- View bids
- View compliance analysis
- Request additional documents
- Review verification status
- Generate reports

5.3 Bidder

Permissions:

- Manage own profile
- View available tenders
- Submit bids
- Upload documents
- View own submissions
- View own compliance status
- Respond to document requests

The bidder must never access another bidder's data.

5.4 Reviewer

Permissions:

- View assigned review cases
- View supporting evidence
- Review AI results
- Override AI result
- Add review comments
- Approve/reject/flag cases

5.5 Auditor

Permissions:

- Read audit logs
- View verification history
- View decision history

- View authorized reports

Auditor access should be read-only.

---

6. Functional Requirements

FR-01 Authentication

The system shall provide secure authentication.

Requirements:

- Login
- Logout
- Password hashing
- Session/token management
- Authentication expiry
- Account status checking
- Unauthorized request rejection

Passwords must never be stored in plaintext.

FR-02 Authorization

The system shall implement backend-enforced RBAC.

Every protected API must verify:

1. User authentication
2. User role
3. Resource ownership/access
4. Required permission

Frontend restrictions alone are insufficient.

FR-03 User Management

Admin shall be able to:

- Create users
- View users
- Assign roles
- Activate/deactivate users
- Reset credentials through secure mechanisms

FR-04 Tender Creation

Procurement Officer shall be able to create a tender containing:

- Tender ID
- Title
- Description
- Organization
- Category
- Submission deadline
- Status
- Tender document
- Additional metadata

Tender statuses may include:

- Draft
- Published
- Closed
- Under Evaluation
- Completed
- Cancelled

FR-05 Tender Document Upload

The system shall allow authorized officers to upload tender documents.

Supported formats should include:

- PDF
- DOCX
- TXT where appropriate
- Image formats where required

The system must validate:

- File extension
- MIME type
- File size
- File integrity

FR-06 Tender Requirement Extraction

The system shall extract requirements from tender documents.

Requirement categories include:

- Eligibility
- Financial
- Technical
- Experience
- Certification
- Registration
- Mandatory documents
- Product specifications
- Quantity
- Deadlines
- Other conditions

Each extracted requirement should contain:

- Requirement ID
- Requirement text
- Category
- Mandatory flag
- Parameters
- Thresholds/operators where applicable
- Source document
- Source page
- Extraction confidence

FR-07 Requirement Normalization

The system shall convert natural-language requirements into structured rules wherever possible.

Example:

"Bidder must have minimum annual turnover of ₹50 lakh."

Structured representation:

```
type = financial
field = annual_turnover
operator = >=
threshold = 5000000
mandatory = true
```

The original requirement text must always be preserved.

FR-08 Bid Submission

A bidder shall be able to submit a bid against an available tender.

A bid should contain:

- Bid ID
- Tender ID
- Bidder ID
- Submission timestamp
- Status
- Submitted documents
- Compliance status

The system must prevent unauthorized modification after the applicable submission deadline unless explicitly permitted by the configured workflow.

FR-09 Document Upload

Bidder shall be able to upload supporting documents.

Examples:

- GST certificate
- PAN
- Registration certificate
- MSME certificate
- Experience certificate
- Financial documents
- Technical certificates
- Authorization certificates
- Other tender-specific documents

Each document must be associated with:

- Bid
- Bidder
- Document type
- Upload timestamp
- Processing status
- Storage identifier

FR-10 Secure File Processing

Uploaded files shall pass through a secure processing pipeline.

Pipeline:

```
Upload
↓
Validation
↓
Secure Storage
↓
Malware/Safety Check where available
↓
Text Extraction/OCR
↓
Document Classification
↓
Data Extraction
↓
Validation
↓
Compliance Processing
```

Files must not be executed as code.

FR-11 OCR

The system shall support OCR for scanned documents.

OCR output shall include, where possible:

- Extracted text
- Page number
- Confidence
- Processing status

OCR failure must not crash the complete application.

FR-12 Document Classification

The system should classify uploaded documents.

Example:

```
GST Certificate
PAN
Experience Certificate
Financial Statement
```

```
Technical Certificate
Other
```

If classification confidence is low, the document should be flagged for review.

FR-13 Entity and Field Extraction

The system shall extract relevant entities.

Examples:

- Company name
- GST number
- PAN
- Registration number
- Certificate number
- Dates
- Financial values
- Address
- Authorized person
- Technical values

Each extracted field should retain:

- Value
- Confidence
- Source document
- Page/evidence location

FR-14 Data Normalization

The system shall normalize extracted information before comparison.

Examples:

- Case normalization
- Whitespace normalization
- Date normalization
- Currency normalization
- Number normalization
- Organization-name normalization

Normalization must not destroy the original extracted value.

FR-15 Compliance Engine

The system shall compare extracted evidence with tender requirements.

Possible outcomes:

COMPLIANT

Evidence satisfies the requirement.

NON_COMPLIANT

Evidence clearly fails the requirement.

NEEDS_REVIEW

Evidence is missing, ambiguous, contradictory or insufficiently reliable.

Every result must contain:

- Requirement
- Evidence
- Decision
- Reason
- Confidence
- Verification status

FR-16 Deterministic Rule Engine

Where a requirement can be evaluated deterministically, use rules instead of relying entirely on an LLM.

Examples:

```
turnover >= required_turnover
certificate_expiry >= submission_date
experience_years >= required_years
required_document exists
technical_value >= minimum_value
```

Rules should be configurable and testable.

FR-17 AI Compliance Analysis

AI may be used for:

- Semantic requirement understanding
- Evidence matching
- Ambiguous document interpretation
- Reason generation
- Summarization

AI output must use structured schemas.

AI must not fabricate evidence.

If evidence cannot be found, the result must indicate that evidence is unavailable.

FR-18 Evidence Traceability

Every compliance decision must be traceable to supporting evidence.

The system should store:

- Document ID
- Page number
- Extracted text/value
- Requirement ID
- Processing method
- Decision
- Timestamp

The user should be able to navigate from a compliance result to the supporting document/evidence where permitted.

FR-19 Cross-Document Verification

The system shall compare information across documents.

Examples:

```
GST Name
vs
PAN Name
vs
Registration Name
```

Check for:

- Name inconsistencies
- Registration number conflicts
- Certificate number conflicts
- Date conflicts
- Financial inconsistencies
- Address inconsistencies

A mismatch should generally produce a flag/review condition rather than automatically claiming fraud.

FR-20 Certificate Validity

The system shall evaluate certificate validity where dates are available.

Example:

```
Certificate expiry date < bid submission date
```

Result:

```
NON_COMPLIANT
```

or:

```
NEEDS_REVIEW
```

depending on the tender rule.

FR-21 Verification Layer

Create a pluggable verification architecture.

Supported verification sources may include:

- Authorized APIs
- Public verification services
- Organization-provided systems
- Mock verification providers for development/demo

The system must clearly distinguish:

```
VERIFIED
UNVERIFIED
VERIFICATION_FAILED
VERIFICATION_UNAVAILABLE
MOCK_VERIFICATION
```

Mock verification must never be represented as actual government verification.

FR-22 Verification Records

Store:

- Verification ID
- Verification type
- Source
- Reference number where applicable
- Timestamp
- Status
- Result
- Evidence
- Error information

FR-23 Confidence and Risk

The system shall calculate/display confidence or risk indicators where appropriate.

Possible factors:

- OCR confidence
- Extraction confidence
- Rule certainty
- Verification result
- Document consistency
- Missing evidence

Low-confidence cases should be routed to human review.

FR-24 Human Review

Create review tasks when:

- AI confidence is low
- Required evidence is missing

- Documents conflict
- Verification fails
- Requirement is ambiguous
- Potential inconsistency is detected

Reviewer shall be able to:

- Accept AI result
- Reject AI result
- Mark needs clarification
- Request additional document
- Add comments

FR-25 Reviewer Override

If a reviewer changes an AI/rule result, store:

- Original decision
- New decision
- Reviewer
- Timestamp
- Reason
- Comment

Reviewer decisions must be auditable.

FR-26 Compliance Dashboard

The system shall provide dashboards containing:

- Total tenders
- Total bids
- Compliant bids
- Non-compliant bids
- Pending reviews
- Missing documents
- Verification failures
- Flagged cases

FR-27 Bidder Dashboard

Bidder shall see:

- Available tenders
- Submitted bids

- Document status
- Missing documents
- Processing status
- Compliance status
- Review/document requests

FR-28 Reviewer Dashboard

Reviewer dashboard shall show:

- Assigned cases
- Pending cases
- High-priority cases
- Low-confidence cases
- Completed reviews

FR-29 Audit Dashboard

Authorized auditors/admins shall be able to view:

- User actions
- Document actions
- Verification history
- Compliance decisions
- Reviewer overrides
- Report generation
- Security-relevant events

FR-30 Report Generation

The system shall generate a final compliance report.

Report should include:

1. Tender details
2. Bidder details
3. Requirement checklist
4. Submitted documents
5. Extracted evidence
6. Compliance decisions
7. Verification results
8. Exceptions
9. Reviewer decisions
10. Final recommendation

11. Audit/reference information

FR-31 Notifications

The system may provide notifications for:

- Document processing completion
- Missing documents
- Review requests
- Verification completion
- Compliance completion
- Important status changes

FR-32 Audit Logging

Log important events including:

- Login
- Logout
- User creation
- Role changes
- Tender creation
- Tender modification
- Document upload
- Document access
- AI processing
- Verification
- Compliance decision
- Reviewer override
- Report generation

Audit record should contain:

```
actor
action
resource
timestamp
result
metadata
```

---

7. Non-Functional Requirements

NFR-01 Security

Security is a core system requirement.

Implement:

- Secure authentication
- Backend RBAC
- Resource-level authorization
- Password hashing
- Secure token handling
- HTTPS-ready deployment
- Encryption in transit
- Encryption at rest where supported
- Secure secrets management
- Input validation
- SQL injection protection
- Rate limiting where appropriate
- Secure file uploads
- Path traversal protection
- Access-controlled document retrieval
- Safe error messages
- Audit logging

NFR-02 Privacy

Sensitive bidder/document information must only be accessible to authorized users.

The system should follow data-minimization principles.

Do not expose sensitive information unnecessarily in:

- Logs
- Error messages
- API responses
- Frontend URLs
- Debug output

NFR-03 Performance

The system should:

- Return normal API responses quickly
- Process documents asynchronously where necessary
- Avoid blocking API requests during long OCR/AI operations
- Provide processing status to users

Long-running jobs should use background workers/job queues where appropriate.

NFR-04 Reliability

Failures in one processing stage should not corrupt the entire bid.

For example:

```
OCR failure
↓
mark document processing failed
↓
notify user/reviewer
↓
allow retry/manual processing
```

NFR-05 Scalability

The architecture should allow future scaling of:

- API servers
- OCR workers
- AI workers
- Document storage
- Database
- Verification services

NFR-06 Maintainability

Use modular architecture.

Separate:

- API
- Authentication
- Database
- AI
- OCR
- Compliance
- Verification
- File management
- Reporting
- Audit

Business logic should not be concentrated inside API route handlers.

NFR-07 Explainability

The system must explain why a compliance decision was generated.

Avoid outputs such as:

```
Result: NON_COMPLIANT
```

without evidence.

Preferred:

```
Requirement:
Minimum turnover ₹50 lakh

Extracted:
₹35 lakh

Evidence:
Financial document, page 4

Result:
NON_COMPLIANT

Reason:
Extracted turnover is below the required threshold.
```

NFR-08 Availability

The system should gracefully handle:

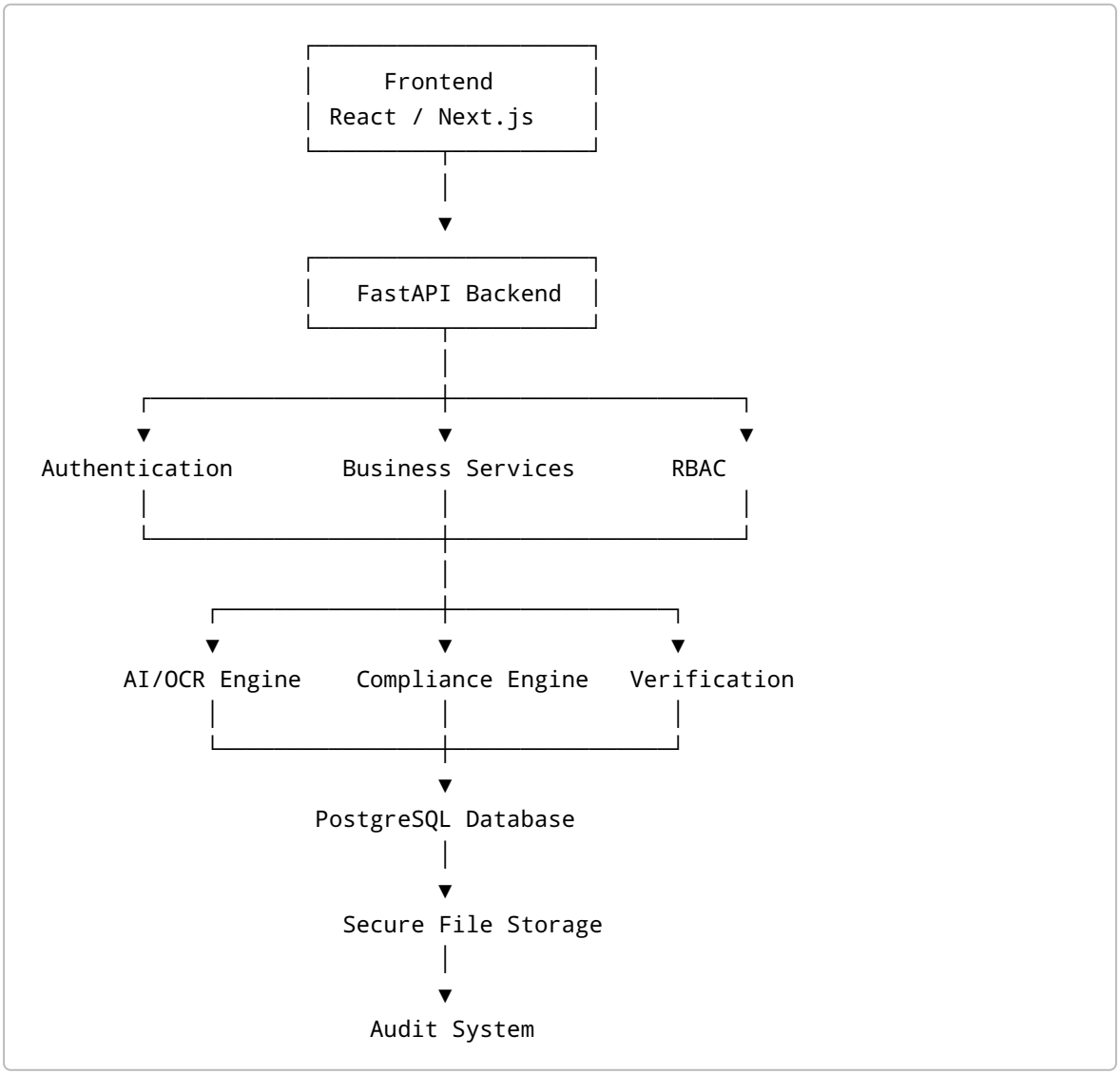
- AI provider failures
- OCR failures
- Verification service failures
- Database errors
- Network failures

Where possible, allow retry and manual review.

---

8. System Architecture

Recommended architecture:



---

9. Recommended Technology Stack

Frontend

- React
- Next.js
- TypeScript
- Tailwind CSS or equivalent UI framework

Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic

Database

- PostgreSQL

AI

- LLM provider abstraction
- Structured AI outputs
- Embeddings/vector search where genuinely useful

OCR

Use a pluggable OCR service/library capable of processing scanned documents.

Background Processing

Use a job queue/background worker architecture for:

- OCR
- AI processing
- Verification
- Report generation

Storage

Use secure object/file storage abstraction.

---

10. Database Requirements

Minimum entities:

```
users
roles
permissions
user_roles

tenders
tender_documents
tender_requirements

bids
bid_documents

document_extractions
extracted_entities

compliance_results
verification_records

review_tasks
review_decisions

notifications
audit_logs
reports
```

Use:

- Primary keys
- Foreign keys
- Unique constraints
- Appropriate indexes
- Created/updated timestamps
- Soft deletion where appropriate

---

11. API Requirements

Example REST API structure:

```
POST   /api/v1/auth/login
POST   /api/v1/auth/logout

GET    /api/v1/users
POST   /api/v1/users

POST   /api/v1/tenders
GET    /api/v1/tenders
GET    /api/v1/tenders/{id}
```

```
PUT    /api/v1/tenders/{id}

POST   /api/v1/tenders/{id}/documents
POST   /api/v1/tenders/{id}/extract-requirements

POST   /api/v1/bids
GET    /api/v1/bids/{id}

POST   /api/v1/bids/{id}/documents
GET    /api/v1/bids/{id}/documents

POST   /api/v1/bids/{id}/process
GET    /api/v1/bids/{id}/compliance

POST   /api/v1/bids/{id}/verify

GET    /api/v1/reviews
GET    /api/v1/reviews/{id}
POST   /api/v1/reviews/{id}/decision

GET    /api/v1/reports/{bid_id}

GET    /api/v1/audit-logs
```

All protected endpoints must enforce authorization.

---

12. Data Flow

Tender Flow

```
Officer
  ↓
Create Tender
  ↓
Upload Tender
  ↓
Document Processing
  ↓
AI Requirement Extraction
  ↓
Requirement Validation
  ↓
Structured Requirements
```

Bid Flow

```
Bidder
  ↓
Select Tender
  ↓
Submit Bid
  ↓
Upload Documents
  ↓
File Validation
  ↓
OCR / Parsing
  ↓
Document Classification
  ↓
Entity Extraction
```

Compliance Flow

```
Structured Requirements
          +
Extracted Bidder Evidence
          ↓
Requirement Matching
          ↓
Rule Engine
          +
AI Analysis
          +
Verification
          ↓
Compliance Result
          ↓
Human Review if Required
          ↓
Final Decision
```

---

13. Security Threat Requirements

The implementation must protect against at least:

- Broken access control
- IDOR/resource-access attacks
- SQL injection

- XSS
- CSRF where applicable
- Path traversal
- Malicious file uploads
- Credential theft
- Secret leakage
- Token misuse
- Excessive API requests
- Unauthorized document access
- Privilege escalation

Example critical requirement:

If bidder A requests:

```
GET /api/v1/bids/BIDDER_B_BID_ID/documents
```

the backend must reject the request unless bidder A has explicit authorization.

Never rely on hiding the ID in the frontend.

---

14. Testing Requirements

Unit Testing

Test:

- Authentication
- Authorization
- Requirement parser
- Rule engine
- Date calculations
- Compliance calculations
- Data normalization
- Verification adapters

Integration Testing

Test:

- Login
- Tender creation
- Tender upload
- Requirement extraction
- Bid submission
- Document processing
- Compliance analysis

- Review workflow
- Report generation

Security Testing

Test:

- Unauthorized access
- Cross-bid document access
- Role escalation
- Invalid tokens
- Malicious uploads
- Injection attempts
- Invalid API parameters

End-to-End Test

The following scenario must work:

```
Admin
↓
Create users

Officer
↓
Create tender
↓
Upload tender
↓
Extract requirements

Bidder
↓
Submit bid
↓
Upload documents

System
↓
OCR
↓
Extract fields
↓
Run compliance engine
↓
Run verification
↓
Generate results

Reviewer
```

```
↓
Review flagged cases
↓
Approve/Reject

System
↓
Generate final compliance report
↓
Store audit trail
```

---

15. Demo Requirements

Provide synthetic demo data.

Demo should include:

Case 1 — Fully Compliant

All mandatory requirements satisfied.

Case 2 — Missing Document

Required certificate not uploaded.

Case 3 — Expired Certificate

Certificate expired before the relevant submission date.

Case 4 — Financial Failure

Required turnover is greater than bidder's extracted turnover.

Case 5 — Data Mismatch

Company information differs across documents.

Case 6 — Needs Review

AI/OCR confidence is low or evidence is ambiguous.

These cases should demonstrate the complete capabilities of the platform.

---

16. Acceptance Criteria

The MVP will be considered successful when:

1. Users can securely log in.
2. Roles and permissions are enforced server-side.
3. Officer can create and upload a tender.
4. System extracts structured requirements.
5. Bidder can submit a bid.
6. Bidder can securely upload documents.
7. System can process digital/scanned documents.
8. Relevant fields can be extracted.
9. Requirements can be matched against evidence.
10. Deterministic compliance rules work correctly.
11. AI can assist with ambiguous/semantic analysis.
12. Every decision has supporting evidence.
13. Cross-document inconsistencies are detected.
14. Verification results are recorded.
15. Low-confidence cases reach a reviewer.
16. Reviewer can override decisions with justification.
17. Final compliance report can be generated.
18. Important actions are recorded in audit logs.
19. Users cannot access unauthorized bidder/tender documents.
20. The complete workflow works using synthetic demo data.

---

17. MVP Definition

The first working version MUST prioritize:

```
Authentication
      ↓
RBAC
      ↓
PostgreSQL
      ↓
Tender Management
      ↓
Tender Requirement Extraction
      ↓
Bid Submission
      ↓
Secure Document Upload
      ↓
OCR / Document Extraction
      ↓
Compliance Engine
      ↓
```

```
Verification
      ↓
Reviewer Workflow
      ↓
Audit Logs
      ↓
Compliance Report
```

Advanced features such as sophisticated anomaly detection, multiple external verification integrations, advanced analytics, and highly autonomous AI agents should be implemented only after the core workflow is stable.

---

18. Final System Principle

The platform should follow this principle:

AI extracts and assists → Rules validate → Verification strengthens evidence → Human reviews uncertainty → System records everything → Final decision remains explainable and auditable.

The final product must be:

Secure + Explainable + Auditable + Modular + Scalable + Human-in-the-loop + SIH Demo Ready.